
Aplicación para la navegación en colecciones de
contenidos multimedia
Application for Browsing Multimedia Content
Collections



Trabajo de Fin de Grado
Curso 2025–2026

Autor

Juan Andrés Hibjan Cardona
Leonardo Prado de Souza

Director

José Luis Sierra Rodríguez

Colaborador

Grado en Ingeniería Informática
Facultad de Informática
Universidad Complutense de Madrid

Aplicación para la navegación en
colecciones de contenidos multimedia
Application for Browsing Multimedia
Content Collections

Trabajo de Fin de Grado en Ingeniería Informática

Autor

Juan Andrés Hibjan Cardona
Leonardo Prado de Souza

Director

José Luis Sierra Rodríguez

Colaborador

Convocatoria: *Junio 2026*

Grado en Ingeniería Informática
Facultad de Informática
Universidad Complutense de Madrid

10 de Junio de 2026

Dedicatoria

...

Agradecimientos

...

Resumen

Aplicación para la navegación en colecciones de contenidos multimedia

Un resumen en castellano de media página, incluyendo el título en castellano. A continuación, se escribirá una lista de no más de 10 palabras clave.

Palabras clave

Máximo 10 palabras clave separadas por comas

Abstract

Application for Browsing Multimedia Content Collections

An abstract in English, half a page long, including the title in English. Below, a list with no more than 10 keywords.

Keywords

10 keywords max., separated by commas.

Índice

1. Introducción	1
1.1. Motivación	1
1.2. Objetivos	2
1.3. Plan de trabajo	2
1.3.1. Fase 1: Investigación y Prototipo en Java (Septiembre 2025 - Enero 2026)	2
1.3.2. Fase 2: Desarrollo Web e Integración (Enero 2026 - Marzo 2026)	3
1.3.3. Cronograma de Ejecución y Diagrama Gantt	3
2. Estado de la Cuestión	5
3. Arquitectura y Metodología	7
3.1. Metodología de Ingeniería del Software	7
3.2. Arquitectura del Sistema	9
3.3. Persistencia y Acceso a Datos	11
4. Desarrollo e Implementación	15
4.1. Representación del contexto de exploración	15
4.1.1. La clase <code>State</code> : filtros y contexto acumulado	15
4.1.2. <code>StateManager</code> : gestión del ciclo de vida del estado	16
4.2. Filtrado clásico y relacional	17
4.2.1. Construcción dinámica de queries SQL	17
4.2.2. Filtros de metadatos	18
4.2.3. Filtros de referencia	18
4.2.4. Facetas: cómputo dinámico de valores disponibles	19
4.3. Navegación transversal	20
4.3.1. El mecanismo de <code>link</code> y <code>goback</code>	20
4.3.2. Subconsultas anidadas: traducción del historial de <i>links</i> a SQL	21
4.4. Operaciones de conjuntos	23
4.4.1. Construcción del <code>unionSet</code>	23
4.4.2. Resolución de la unión en base de datos	24
4.5. Obstáculos encontrados y soluciones implementadas	24

5. Pruebas y Validación	29
5.1. Casos de uso validados	29
6. Conclusiones y Trabajo Futuro	31
6.1. LLM como método de navegación	31
6.2. Distribuir el motor en varias máquinas	31
6.3. Cosas genéricas (usuario, historial, etc.)	31
Introduction	33
Conclusions and Future Work	35
Contribuciones Personales	37
Bibliografía	39
A. Título del Apéndice A	41
B. Título del Apéndice B	43

Índice de figuras

1.1. Diagrama Gantt TFG	4
-----------------------------------	---

Índice de tablas

Introducción

“Sólo hay una victoria decisiva: la última”
— Carl Von Clausewitz

...

1.1. Motivación

En los últimos tiempos hemos pasado por un gran cambio en el paradigma de consumo digital. El acúmulo cuantitativo de contenidos digitales ha sobrepasado su umbral cualitativo y hoy supone una manera distinta de relacionarse con ello. La abundancia de información genera una "sobrecarga de información" (o *information overload*), en la cual todo y cualquier usuario necesita de una cantidad cada vez mayor de tiempo para navegar. Eso se hace especialmente evidente en las plataformas digitales, en particular las de streaming multimedia (como Netflix, Disney+ y HBO Max), en las que difícilmente encontramos qué consumir sin invertir un tiempo sustancial en ello.

Ese cambio de paradigma, y el subsecuente problema, sigue siendo abordado con métodos tradicionales como: la búsqueda directa (por palabras clave), el filtrado clásico por género o medio y los sistemas de recomendación basados en algoritmos ocultos al consumidor. Estos abordajes son limitados y sirven poco al nuevo contexto de consumo digital. La búsqueda directa es determinista y no permite ninguna flexibilidad, especialmente al suponer que el usuario sabe exactamente lo que está buscando de antemano. El filtrado tradicional suele ser incompleto e inconsistente, usando taxonomías ocultas e inescrutables. Los algoritmos de recomendación suelen servir más como un propagador de la burbuja de filtros (o *filter bubble*), al depender directamente del historial de consumo, o de lo que la plataforma desee promover, quitando la agencia del usuario en última instancia.

Este problema, entretanto, no es exclusivo de experiencia de usuario, sino también supone un problema tecnológico relacionado con la forma en la que se diseñan y utilizan las colecciones de información. Los modelos relacionales de navegación y los sistemas de navegación sobre colecciones (*collection navigation systems*), traen un enfoque distinto en el que los datos no se representan como elementos aislados, sino como entidades conectadas mediante relaciones semánticas. Ese abordaje permite

recoger colecciones de manera interactiva, con mecanimos de navegación más expresivos en los que el usuario puede explorar progresivamente una colección compleja combinando filtros, relaciones y transiciones entre entidades.

Existe, por lo tanto, una necesidad declarada de que los sistemas de navegación acompañen ese cambio de paradigma. Eso solo se puede cumplir permitiendo a los usuarios herramientas de exploración interactiva más ricas y flexibles. Dichas herramientas deben combinar la precisión establecida por la búsqueda directa, pero con una mayor expresividad que el filtrado clásico sin quitar la agencia del usuario.

1.2. Objetivos

Descripción de los objetivos del trabajo.

1.3. Plan de trabajo

Hemos optado por una metodología ágil, basada en su mayor parte en el marco de trabajo **Scrum**, adaptada a un proyecto en pareja. La adopción de ese método viene dada por la naturaleza exploratoria e innovadora del proyecto, ya que los requisitos y el motor de navegación necesitaban un refinamiento y optimización iterativos.

El desarrollo fue organizado mediante *sprints* (ciclos de trabajo entre 2 y 3 semanas), definiendo un backlog inicial (*Product Backlog*) con los requisitos fundamentales del sistema. Además, se realizarían reuniones periódicas de seguimiento (*Sprint Reviews*), que permiten evaluar cada incremento del software, adaptar la planificación de acuerdo con los problemas y obstáculos encontrados, en conjunto con la tutoría del profesor.

Con respecto a la macroplanificación, el proyecto se estructuró en dos grandes fases secuenciales de desarrollo:

1.3.1. Fase 1: Investigación y Prototipo en Java (Septiembre 2025 - Enero 2026)

Con el objetivo de validar completa y rigurosamente la viabilidad del motor de navegación y los algoritmos involucrados, se optó por desarrollar un prototipo en Java. Deliberadamente abandonando la implementación inicial de una interfaz gráfica, hemos centralizado los esfuerzos en su eficacia, eficiencia y robustez técnica de los componentes del sistema. Este enfoque permitió analizar en profundidad el comportamiento del algoritmo planteado y la correcta integración con la base de datos TMDB.

- **Estudio precedente y proceso de diseño arquitectónico:** Es ese momento hemos definido es estado de la cuestión basándonos en artículos de investigación previos, seleccionamos los datasets pertinentes y de dominio público y diseñamos el modelo básico de datos relacional.

- **Desarrollo del motor de navegación (núcleo backend):** Siguiendo con la implementación de la lógica en Java, desarrollamos una máquina de estados (*State*, *StateManager*) con capacidad para: gestionar el historial, los filtros de metadatos, los filtros relacionales y los saltos transversales *Link*.

- **Integración de TMBD:**

Para empezar la validación del motor de navegación, hemos optado por integrar la base de datos pública TMBD (*The Movie Database*), que permite obtener información sobre películas, series y personas. Además de consumir la API, se diseñó un proceso de transformación y almacenamiento de los datos en nuestro modelo relacional, permitiendo adaptar la estructura de TMDb a las necesidades de navegación del motor.

1.3.2. Fase 2: Desarrollo Web e Integración (Enero 2026 - Marzo 2026)

Con el núcleo del modelo de navegación funcional, hemos trasladado el esfuerzo en exponer sus funcionalidades a través de una arquitectura cliente-servidor, además de añadir una interfaz de usuario.

- **Exposición de API RESTful:** Hemos desarrollado una capa de controladores usando Servlets de Java (*API RESTful*) para exponer, con seguridad y de manera estructurada, las capacidades del motor (*FacetsServlet*, *NavigationServlet*, *UnionServlet*, etc.) que gestiona el contexto mediante sesiones de usuario.
- **Desarrollo del Frontend:**

Trás definir la API, se desarrolló una interfaz web para la interacción con el motor de navegación. Dentre sus principales componentes están: el área de resultados, sistema de filtrado dinámico de metadatos, controles de navegación de estados, conjunto de controles de filtros y exploración de relaciones. Todo desde el paradigma de API RESTful con actualización dinámica.
- **Pruebas de Integración y Refinamiento:** Unimos todos los componentes, hemos hecho pruebas de carga con los *datasets* completos y evaluado la experiencia de usuario (*UX*).

1.3.3. Cronograma de Ejecución y Diagrama Gantt

- **Septiembre 2025:** Definición del proyecto, revisión de la literatura (estado del arte) y selección de *datasets*.
- **Octubre 2025:** Diseño del modelo de datos y desarrollo inicial del motor de navegación (Fase 1).
- **Noviembre 2025:** Implementación completa de la máquina de estados e historial de navegación.

- **Diciembre 2025:** Prototipado de la integración del dataset TMDB y cierre de la Fase 1.
- **Enero 2026:** Diseño e implementación de la API RESTful (controladores y enrutamiento) (Fase 2).
- **Febrero 2026:** Desarrollo del cliente web (Frontend) e integración con el backend.
- **Marzo 2026:** Pruebas de integración, resolución de bugs (obstáculos técnicos) y validación de casos de uso.
- **Abril 2026:** Redacción intensiva de la memoria del Trabajo de Fin de Grado.
- **Mayo 2026:** Revisiones finales de la memoria, preparación de la presentación, ensayo y defensa ante el tribunal universitario.

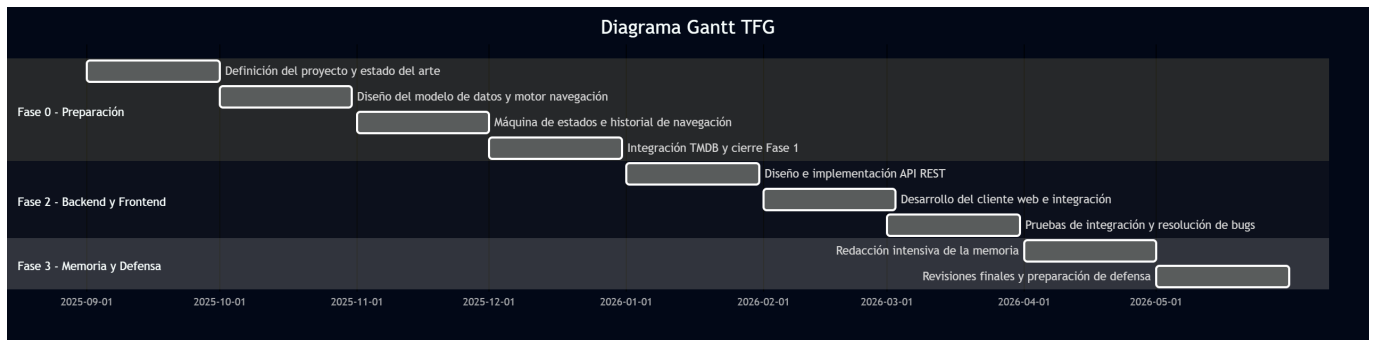


Figura 1.1: Diagrama Gantt TFG

Estado de la Cuestión

En el estado de la cuestión es donde aparecen gran parte de las referencias bibliográficas del trabajo. Una de las formas más cómodas de gestionar la bibliografía en $\text{\LaTeX}$ es utilizando **bibtex**. Las entradas bibliográficas deben estar en un fichero con extensión *.bib* (con esta plantilla se proporciona el fichero *biblio.bib*, donde están las entradas referenciadas más abajo). Cada entrada bibliográfica tiene una clave que permite referenciarla desde cualquier parte del texto con los siguiente comandos:

- Referencia bibliografica con cite: Bautista et al. (1998)
- Referencia bibliográfica con citep: (Oetiker et al., 1996)
- Referencia bibliográfica con citet: Krishnan (2003)

Es posible citar más de una fuente, como por ejemplo (Mittelbach et al., 2004; Lamport, 1994; Knuth, 1986)

Después, $\text{\LaTeX}$ se ocupa de rellenar la sección de bibliografía con las entradas **que hayan sido citadas** (es decir, no con todas las entradas que hay en el *.bib*, sino sólo con aquellas que se hayan citado en alguna parte del texto).

Bibtex es un programa separado de latex, pdflatex o cualquier otra cosa que se use para compilar los *.tex*, de manera que para que se rellene correctamente la sección de bibliografía es necesario compilar primero el trabajo (a veces es necesario compilarlo dos veces), compilar después con bibtex, y volver a compilar otra vez el trabajo (de nuevo, puede ser necesario compilarlo dos veces).

Capítulo 3

Arquitectura y Metodología

Este capítulo centraliza el conjunto de decisiones de diseño técnico y estructural que sirven como fundamentos del proyecto. En primer lugar, exponemos los marcos metodológicos utilizados para el manejo del propio ciclo de vida del software, definiendo las herramientas, prácticas y técnicas de ingeniería aplicadas. Luego, seguimos con el desglose de la arquitectura del sistema, delimitando expresamente la separación de responsabilidades a través de las distintas capas de la aplicación: la capa de control y exposición de la API, la capa de persistencia y acceso a datos, y finalmente, el mecanismo de integración con la interfaz de usuario. El principal objetivo de esta arquitectura fue garantizar un sistema modular, escalable, mantenible y flexible en la integración de base de datos.

3.1. Metodología de Ingeniería del Software

Este Trabajo de Fin de Grado se ha organizado por principios de Ingeniería de Software fundamentados en el paradigma ágil. Como se estableció en el plan de trabajo, hemos optado por **Scrum** como referencia, adaptando sus características a las particularidades de un equipo reducido de dos personas. La adaptación resultó esencial para mantener la disciplina y estructura ofrecida por el modelo ágil sin introducir una sobrecarga innecesaria.

Por eso la aplicación de **Scrum** se centró principalmente en ciclos de desarrollo iterativos (*sprints*) de corta duración, donde se planteaban objetivos concretos y alcanzables. En general, toda iteración coincidía con un incremento funcional del sistema, con excepción de las fases iniciales, en lo cual la mayor parte del esfuerzo se mantuvo en una especificación inicial del motor de navegación. Este abordaje permitió ejecutar una validación progresiva de los componentes críticos del proyecto. Específicamente, se ha notado un gran impacto en los *milestones*: motor de navegación en Java, la integración con la base de datos TMDB y la posterior exposición de funcionalidades mediante una API REST.

Desarrollo iterativo e incremental

A cambio de definir completa y decididamente la arquitectura desde del principio con un enfoque lineal y secuencial, como la metodología cascada, el sistema se ha desarrollado de forma incremental. En sus primeras iteraciones, hemos priorizado la validación del propio modelo de navegación y la estructura de datos adyacente, al mismo tiempo que en fases posteriores hemos incorporado progresivamente la capa de acceso a datos, la API y el *frontendweb*. Dicho tratamiento resultó significativamente relevante debido a la naturaleza exploratoria del proyecto, ya que permitió adaptar el diseño del motor de navegación al mismo paso en que se probaba con datos reales.

Gestión de requisitos y backlog

Las funcionalidades más complejas iniciales, como el sistema de filtrado relacional, la gestión de estados de navegación y la combinación de resultados, se desarrollaron mediante periodos de trabajo intensivo centrados en la implementación y validación de cada componente. En cambio, el resto de tareas de menor complejidad o con menor dependencia técnica se fueron abordando progresivamente siguiendo la prioridad establecida en el *backlog*.

Los criterios de priorización y la planificación de las tareas se realizaron en coordinación con la tutoría del proyecto, llevando a cabo revisiones periódicas y ajustes continuos. De esta forma, los objetivos de cada iteración se adaptaban en función de los resultados obtenidos y de los obstáculos técnicos que iban apareciendo durante el desarrollo.

Trabajo colaborativo en un equipo reducido

Hemos adaptado las prácticas de **Scrum** a un entorno de colaboración directa y continua, ya que el proyecto sería desarrollado por un equipo de dos personas. En lugar de llevar a cabo reuniones formales, se optó por mantener una comunicación frecuente entre los integrantes, manteniendo siempre una división clara y acordada de responsabilidades, fundamental para la paralelización del desarrollo sin perder coherencia en el diseño global. Este método de comunicación y organización facilitó una evolución más eficiente y dinámica del software, con una alta capacidad de reacción frente los problemas técnicos.

Control de versiones y gestión del código fuente

En virtud de la necesidad de mantener la integridad del código y facilitar el trabajo colaborativo, hemos utilizado Git como sistema de control de versiones distribuido. El repositorio principal se alojó en GitHub y se empleó una estrategia de ramificación sencilla : una rama principal (**main**) para el código estable y funcional, y ramas de características (*feature branches*) para el desarrollo de nuevos componentes antes de su integración final. Con esa metodología, hemos podido trabajar en paralelo sin conflictos significativos manteniendo siempre una versión funcional

del sistema.

Buenas prácticas de programación y diseño

Durante el desarrollo hemos dado especial atención a la aplicación de buenas prácticas de programación y diseño software. Se siguieron principios de separación de responsabilidades (*Separation of Concerns*) y modularidad, lo está reflejado en la organización del código en distintos paquetes (por ejemplo, `controller`, `model`, `dao`, `filter`, etc.). Dicha estructura tiende a facilitar la legibilidad del código, su mantenimiento y la posibilidad de ampliación futura sin necesidad de realizar modificaciones significativas o profundas en su arquitectura.

3.2. Arquitectura del Sistema

Para permitir la exposición de la lógica del motor de exploración y su respectivo consumo por parte de aplicaciones, hemos diseñado una arquitectura fundamentada en el modelo Cliente-Servidor. Desde el lado del servidor (*backend*), la aplicación favorece el desacoplamiento al basarse en capas bien definidas. Esta sección se dedica a detallar la capa de control, responsable por enrutar las distintas peticiones HTTP, y la capa de filtros, encargada de controlar las políticas de acceso y seguridad.

Diseño de la API y Controladores

Hemos optado por implementar la interfaz de comunicación entre el cliente web y el motor de navegación usando una API de estilo REST, siguiendo la especificación de *Servlets* de Java (`HttpServlet`). Esta elección viene de priorizar un control directo sobre las peticiones y respuestas HTTP frente al uso de frameworks web de más alto nivel (como Spring Boot), que aunque facilitan el desarrollo de aplicaciones web tradicionales, introducen una mayor complejidad estructural y una capa adicional de abstracción que no resulta necesaria para los objetivos de este proyecto.

El uso directo de `HttpServlet` ha sido necesaria para implementar una capa de comunicación más compacta, donde cada controlador se encarga exclusivamente de recibir los parámetros de la petición, delegar la lógica en el motor de navegación y generar la respuesta correspondiente en formato JSON. Emplear esa estrategia fue importante para mantener una separación explícita entre la lógica de navegación y la capa de acceso web. Además permitió un control más preciso de la gestión de sesiones, el formato de las respuestas y el flujo de las peticiones HTTP.

Para ejecutar la aplicación se ha utilizado Apache Tomcat como contenedor de *Servlets*. Nos hemos inclinado por Tomcat frente a otras opciones principalmente por la naturaleza del proyecto. Tomcat es un contenedor considerablemente leve, enfocado en la ejecución de aplicaciones basadas en *Servlets* y JSP, lo que lo convierte en una solución adecuada para trabajar directamente con la API de Java sin generar dependencias adicionales.

Por motivos semejantes, hemos descartado servidores de aplicaciones completos como GlassFish, ya que requiere tecnologías adicionales como EJB y gestión

avanzada de recursos. Alternativas como Jetty también permiten ejecutar *Servlets*, pero está más orientado a una integración embebida dentro de otras aplicaciones/frameworks. Un contenedor independiente, sencillo de configurar y con respaldo de experiencia en entornos académicos y de desarrollo hace que Tomcat resulte la opción más estable encontrada.

Como contrapartida, esta decisión implica un mayor esfuerzo de implementación manual, especialmente en tareas como el enrutamiento de peticiones, la serialización de datos o la gestión de errores. Sin embargo, este *trade-off* se ha asumido de forma consciente, ya que el objetivo principal del proyecto no es el desarrollo de una aplicación web en sí, sino el diseño e implementación de un motor de navegación sobre colecciones de datos, siendo la API web únicamente un mecanismo para exponer dicha funcionalidad.

Gestión del Estado de Navegación (Enfoque *Stateful*)

El manejo del contexto del usuario no es trivial, ya que, a diferencia de aplicaciones web tradicionales basadas en peticiones independientes, la naturaleza de nuestro proyecto implica una navegación que consiste en una exploración progresiva, donde cada acción del usuario modifica un estado acumulativo que condiciona las operaciones posteriores.

Las arquitecturas REST normalmente trabajan con sistemas *stateless* (sin estado), donde cada petición del cliente debe contener toda la información necesaria para ser procesada. En nuestro caso, dicha naturaleza es altamente iterativa y acumulativa, de manera que hacer que cada petición esté completamente encapsulada y atomizada puede suponer un problema.

Requerir que el cliente envíe en cada petición el historial completo de navegación, las listas de filtros aplicados, las operaciones realizadas y los saltos relacionales supondría una sobrecarga de red inasumible. Este estado no puede considerarse independiente entre peticiones, sino que constituye una estructura dinámica que evoluciona a lo largo de la sesión de usuario. Además, requerir la petición completa añadiría una complejidad excesiva en el *frontend*, aumentando el tamaño de cada *request*, empeorando el rendimiento y dificultando considerablemente la escalabilidad.

Por este motivo, se ha optado por un diseño *stateful*, en el que el contexto de navegación se mantiene en el servidor. Al inicio de la interacción, se crea una sesión de usuario (**HttpSession**) en la que se almacena una instancia de **StateManager**, encargada de gestionar el estado actual y las transiciones entre estados del usuario.

De este modo, los distintos controladores pueden operar sobre un contexto compartido y persistente, aplicando las acciones del usuario (como añadir filtros, explorar relaciones o combinar resultados) de forma centralizada en el *backend*. Este enfoque permite simplificar la lógica del cliente, mejorar la eficiencia de las comunicaciones y mantener una coherencia estricta con el modelo de navegación propuesto.

Componentes transversales de la arquitectura

El sistema contiene una serie de componentes transversales, para allá de la capa de controladores, que proporcionan funcionalidades comunes que permiten separar claramente la lógica de dominio y los aspectos técnicos propios de la infraestructura web.

Como mencionado en la sección anterior, la gestión de sesiones es un elemento fundamental dentro de la arquitectura, pues es el responsable por mantener el contexto de navegación del usuario entre peticiones distintas. A través del mecanismo de `HttpSession`, el sistema asocia cada usuario con una instancia de `StateManager`, lo que garantiza la persistencia del estado de navegación y posibilita un modelo de interacción basado en estados.

Por otro lado, la serialización de datos en formato JSON empleada de manera sistemática funciona como el mecanismo clave para el intercambio de información entre el backend y el frontend. Este abordaje se ha hecho necesario principalmente para desacoplar ambas capas, lo que facilita la evolución independiente de la interfaz de usuario y del motor de navegación como tal.

Adicionalmente, hemos implementado un filtro encargado de gestionar aspectos técnicos de la comunicación HTTP, como el intercambio de recursos entre distintos orígenes - política CORS (*Cross-Origin Resource Sharing*). Este componente intercepta las peticiones entrantes antes de que alcancen los controladores y añade las cabeceras necesarias para permitir la comunicación entre cliente y servidor cuando se ejecutan en dominios o puertos distintos, necesario para evitar las restricciones por defecto de seguridad del navegador. De este modo, se evita introducir lógica técnica adicional en los controladores y se mantiene su enfoque en la lógica de dominio.

3.3. Persistencia y Acceso a Datos

La capa de persistencia y su respectivo diseño constituyen la base fundamental de la arquitectura del sistema, puesto que el motor de navegación tiene una relación de dependencia directa de la estructura y disponibilidad de los datos sobre los que opera. Como el sistema no se limita a recuperación de la información, este proyecto los datos no se consumen directamente, sino que pasan por una transformación y reorganización que permiten la exploración mediante un modelo de navegación relacional.

Para posibilitar dicha exploración y mantener un nivel razonable de independencia del dominio, hemos decidido desacoplar completamente la lógica del motor de las fuentes de datos concretas. Para tanto, hemos definido un modelo interno de representación que abstrae las entidades y relaciones relevantes del dominio, permitiendo trabajar de forma uniforme independientemente del origen de los datos. Este enfoque es particularmente relevante dado que el sistema integra conjuntos de datos heterogéneos, los cuales deben ser "traducidos" bajo una misma estructura conceptual.

Por esa misma razón, hemos optado por organizar el acceso a datos a través de una capa dedicada de abstracción (*DAO*), que se encarga de aislar la lógica

de acceso, traducción y recuperación de la información. Esta separación es lo que garantiza mantener la independencia entre la lógica de navegación y los mecanismos de persistencia, permitiendo al mismo tiempo una evolución del propio sistema y posibles cambios en las fuentes de datos o incluso en su forma de almacenamiento.

Modelo de datos y representación interna

El sistema se basa en un modelo de datos interno diseñado específicamente para soportar la navegación sobre colecciones complejas, permitiendo representar relaciones cualitativamente distintas independiente del dominio asociado. Decidimos no operar directamente sobre las estructuras proporcionadas por las fuentes de datos externas, y sí definir una representación propia basada en entidades y relaciones, adaptada a las necesidades del motor de navegación.

En dicho modelo, los elementos del dominio se representan como entidades independientes, caracterizadas por un conjunto de atributos (**metadatos**), mientras que las interacciones entre dichas entidades se expresan mediante relaciones explícitamente manifiestas. Esta separación permite tratar de forma homogénea tanto el filtrado por atributos (por ejemplo, características propias de una entidad) como la exploración relacional (navegación entre entidades conectadas).

Al mismo tiempo, esta representación actúa como una capa de abstracción sobre las fuentes de datos, unificando conjuntos heterogéneos bajo una estructura común e independiente de su formato u origen. De este modo, el motor de navegación puede operar de forma desacoplada, garantizando tanto la extensibilidad del sistema como la coherencia en las operaciones realizadas sobre los datos.

Integración y adquisición de datos

Dado que el sistema utiliza y transforma fuentes de datos heterogéneas, una de las decisiones arquitectónicas principales ha sido definir un proceso de integración que permita desacoplar la adquisición de datos de su representación interna. En lugar de depender directamente de la estructura y organización de cada fuente, se ha establecido un mecanismo de transformación que adapta los datos externos al modelo común basado en entidades y relaciones descrito anteriormente.

Este proceso de integración implica la extracción de la información relevante de cada fuente de datos, su normalización y su posterior adaptación a las estructuras internas del sistema. De este modo, se garantiza que, independientemente del formato, granularidad o semántica original de los datos, todos ellos puedan ser tratados de forma uniforme por el motor de navegación.

Una de las consideraciones más importantes en este diseño ha sido la coexistencia de distintos conjuntos de datos con características complementarias. Antes de privilegiar una única fuente como referencia principal, hemos diseñado el sistema para permitir la incorporación de nuevas fuentes, siempre que estas puedan ser mapeadas al modelo interno. Este enfoque posibilita la extensibilidad del sistema y evita dependencias estrictas con tecnologías o fuentes concretas.

Asimismo, se ha optado por un modelo de integración en el que la responsabilidad de adaptación recae en la capa de acceso a datos, evitando trasladar esta complejidad

al motor de navegación. De este modo, el núcleo del sistema opera exclusivamente sobre estructuras homogéneas, manteniendo una separación clara entre la lógica de dominio y los detalles específicos de cada fuente de datos.

Capa de acceso a datos (DAO)

Para mantener una separación eficiente entre la lógica de navegación y los mecanismos de acceso a datos, hemos definido una capa específica de abstracción basada en el patrón *Data Access Object* (DAO). La capa actúa como intermediaria entre el modelo interno del sistema y las distintas fuentes de datos, encapsulando toda la lógica necesaria para la recuperación y adaptación de la información.

La principal responsabilidad de los componentes DAO es proporcionar una interfaz uniforme de acceso a los datos, independiente de su origen. De este modo, el motor de navegación no interactúa de manera directa con las fuentes externas, sino que opera sobre estructuras ya adaptadas al modelo interno. Hemos tomado esta decisión para aislar completamente el núcleo del sistema de detalles específicos (como el formato de los datos, los mecanismos de acceso o mismo las particularidades de cada fuente).

Además, esta capa facilita la integración de múltiples conjuntos de datos, ya que cada fuente puede ser gestionada mediante su propia implementación de acceso, de manera completamente independiente, sin afectar al resto del sistema. Esto resulta especialmente relevante en un contexto donde hay una coexistencia de datos heterogéneos, permitiendo su incorporación de manera progresiva, controlada y sin grandes dependencias.

Esta decisión también es relevante para hacer la mantenibilidad y evolución del sistema de manera flexible. Al centralizar la lógica de acceso a datos en una capa específica, es posible realizar cambios en las fuentes de información, en sus formatos o en las estrategias de recuperación sin necesidad de modificar el motor de navegación o la capa de control. Por lo tanto, se garantiza una arquitectura modular, robusta, preparada para futuras extensiones y la incorporación de otras colecciones de datos.

Estrategias de gestión de datos

Como mencionado anteriormente, hemos priorizado el diseño del sistema a partir de una separación clara entre dos niveles de gestión de datos: el estado de navegación del usuario y los datos del dominio. Esta distinción responde a criterios distintos de persistencia, mutabilidad y coste de acceso, y determina la arquitectura general del sistema.

Los datos del dominio — entidades, metadatos y relaciones — se almacenan de forma persistente en una base de datos relacional. Esta decisión facilita que el sistema soporte datasets de tamaño potencialmente grandes sin imponer restricciones sobre la memoria disponible, y delega en el motor de base de datos la responsabilidad de ejecutar las operaciones de filtrado y recuperación de forma razonable.

El estado de navegación, en cambio, es ligero, momentáneo y específico de cada usuario: recoge los filtros acumulados, el historial de transiciones entre colecciones y los conjuntos de resultados en construcción. Hemos decidido mantenerlo en memoria

dentro de la sesión HTTP por su naturalidad, dado su ciclo de vida acotado y su reducido tamaño, además de evitar serializar y deserializar el contexto de navegación en cada petición.

La consecuencia más relevante de esta separación es que el sistema no materializa subconjuntos intermedios de resultados: en cada petición, el estado en memoria se traduce directamente a una consulta sobre la base de datos. Esto simplifica el modelo de datos a costa de trasladar la complejidad al proceso de construcción dinámica de queries, que debe reflejar fielmente el contexto de navegación acumulado. Esta tensión — entre simplicidad del modelo y complejidad de las consultas — es una de las decisiones de diseño que hemos evaluado tener un gran impacto en la implementación, y se aborda en detalle en el capítulo siguiente.

Desarrollo e Implementación

4.1. Representación del contexto de exploración

Para comprender las decisiones de diseño del modelo de representación interno, es útil partir de un ejemplo concreto: la estructura del dataset de The Movie Database (TMDb), que fue la primera y principal fuente de datos utilizada durante el desarrollo. TMDb organiza su información en torno a cinco tipos de entidades: películas, series de televisión, personas, productoras y cadenas de televisión. Cada tipo de entidad posee atributos propios — como el género o la fecha de estreno en el caso de las películas, o el género y la fecha de nacimiento — y mantiene relaciones directas con entidades de otros tipos. Así, una película puede estar producida por una o varias productoras, contar con actores y miembros de equipo técnico de la colección de personas, una serie puede estar asociada a una cadena de televisión concreta, etc. O sea, al final tenemos una base de datos suficientemente compleja, con atributos específicos al dominio de cada entidad, y posibles relaciones de navegación, con aristas que conectan entidades de igual o distinta naturaleza.

Estas relaciones son bidireccionales en el dataset procesado: si una película referencia a una persona como *Director*, esa persona referencia a su vez a la película bajo la razón *Director*. Esto permite recorrer el grafo de relaciones en ambas direcciones desde cualquier punto de partida.

Al analizar esta estructura se infirieron las dos abstracciones fundamentales del modelo interno. Por un lado, los **metadatos**: atributos propios de cada entidad, representados como pares clave-valor, sobre los que el usuario puede establecer condiciones de inclusión o exclusión. Por otro lado, las **referencias**: vínculos entre entidades de distintas colecciones, etiquetados con un tipo de relación (*reason*), que permiten tanto filtrar por asociación como navegar de una colección a otra siguiendo esos vínculos. Esta distinción — metadatos para describir, referencias para relacionar — es la que articula toda la representación del contexto de exploración.

4.1.1. La clase State: filtros y contexto acumulado

La clase `State` representa el contexto de exploración de un usuario en un momento dado. Encapsula, por un lado, los filtros activos sobre la colección actual y,

por otro, el historial de transiciones realizadas entre colecciones durante la sesión actual.

Los filtros se organizan en cuatro estructuras `HashMap` anidadas. Los filtros de metadatos (`mfilters`) mapean cada atributo a un conjunto de valores seleccionados, expresando que las entidades resultantes deben satisfacer al menos uno de dichos valores por atributo. Su contraparte negativa (`notMfilters`) excluye las entidades que coincidan con los valores indicados. Los filtros de referencia (`rfilters`) operan sobre relaciones entre colecciones: para cada colección referenciada y tipo de relación, almacenan el conjunto de identificadores de entidades con los que debe existir vínculo. De igual modo, `notRfilters` excluye las entidades que presenten dichos vínculos. Todos estos mapas están indexados por el identificador de la colección activa, de forma que el mismo objeto `State` puede acumular filtros correspondientes a distintas colecciones visitadas a lo largo de la navegación.

Junto a los filtros, `State` mantiene una lista de objetos `Link`. Cada `Link` registra una transición entre colecciones: almacena la dirección de la transición (bidireccional), la colección de origen, el tipo de relación seguida y una copia del estado en el momento de realizar la transición. Esta lista constituye el historial de navegación y es lo que permite reconstruir, a la hora de ejecutar una consulta, el camino completo que el usuario ha recorrido entre colecciones.

`State` implementa `Serializable` y facilita un constructor de copia que realiza una copia profunda de todas sus estructuras internas, necesaria para preservar instantáneas del estado en distintos puntos de la sesión.

4.1.2. `StateManager`: gestión del ciclo de vida del estado

`StateManager` es el punto de entrada a través del cual los controladores interactúan con el estado de navegación. Se instancia al inicio de la sesión, cuando el usuario selecciona un dataset y una colección de partida, y se almacena como atributo de la sesión HTTP bajo la clave `navState`.

Internamente, `StateManager` coordina tres estructuras con ciclos de vida distintos. La primera es el estado activo (`cur`), que representa el contexto de exploración en curso y sobre el que se aplican todas las operaciones de filtrado y navegación. La segunda es una pila de historial (`historyStack`), una `Deque` que el `NavigationServlet` alimenta invocando `save()` antes de cada acción, lo que permite revertir el estado mediante `restore()`. La tercera es el `unionSet`, una lista de estados congelados que acumula los contextos que el usuario ha marcado para combinar mediante unión; al invocar `union()`, el estado activo se añade al `unionSet` y `cur` se reinicializa con un estado limpio para continuar la exploración de manera independiente.

Es responsabilidad íntegra de `StateManager` la gestión de coherencia entre estas tres estructuras. Por ejemplo, al ejecutar `link()` o `union()`, es `StateManager` quien decide cuándo realizar una copia profunda del estado activo antes de cambiarlo, garantizando que los estados almacenados en el historial y en el `unionSet` no se vean afectados por operaciones posteriores. La API pública de `StateManager` refleja el vocabulario de acciones disponibles desde el frontend — `addMetadataFilter`, `removeReferenceFilter`, `link`, `goback`, `save`, `restore` y `union` — delegando en

State las operaciones sobre filtros, y aplicando directamente la lógica de coordinación cuando la operación afecta a más de una de sus estructuras internas.

4.2. Filtrado clásico y relacional

En TMDb, la exploración de un dataset raramente se agota filtrando por atributos propios de una colección, por cuenta del tamaño y complejidad de la colección. Un caso de uso habitual es, por ejemplo, encontrar películas de un género concreto dirigidas por una persona específica, o excluir series producidas por una determinada compañía. El primer tipo de condición opera sobre los metadatos de la entidad; el segundo, sobre sus relaciones con entidades de otra colección. Esta distinción entre *filtrado clásico* — sobre atributos — y *filtrado relacional* — sobre vínculos — motivó el diseño de dos mecanismos de filtrado independientes y de naturaleza distinta pero que se aplican de forma conjunta en cada consulta.

Ambos mecanismos admiten además una variante por la negativa: es posible no solo incluir entidades que cumplan alguna condición, sino también excluir explícitamente las que la cumplan. La combinación de inclusiones y exclusiones, tanto sobre metadatos como sobre referencias, define el espacio de resultados que el usuario está explorando en cada momento.

4.2.1. Construcción dinámica de queries SQL

El núcleo de la capa de acceso a datos es la traducción del estado de navegación a una query SQL ejecutable. Esta traducción se ensambla de manera incremental: en cada petición, el `NavigationDAO` construye la query de forma incremental añadiendo cláusulas `AND` sobre una consulta base que selecciona entidades de la colección activa.

El `appendFilterClauses` es el método central de este proceso, que recibe el `StringBuilder` de la query en construcción, la lista de parámetros y el estado actual, y le añade todas las condiciones derivadas de los filtros acumulados. Para el caso más simple — sin historial de navegación entre colecciones — delega directamente en `appendStateFilters`, que itera sobre los cuatro mapas de filtros del estado (`mfilters`, `notMfilters`, `rfilters`, `notRfilters`) y genera una subconsulta `EXISTS` o `NOT EXISTS` por cada condición activa. Cuando el estado contiene además un historial de *links*, `appendFilterClauses` actúa de forma recursiva, generando subconsultas anidadas que incorporan el contexto de cada transición previa; este mecanismo se describe en detalle en la sección 4.3.

Todas las queries se construyen con parámetros posicionales (`?`), nunca interpolando valores directamente en el SQL. Los valores se acumulan en una lista `params` y se asignan posteriormente mediante `stmt.setObject()`, lo que delega en el `PreparedStatement` de JDBC la responsabilidad de escapar los valores antes de enviarlos al motor, para prevenir inyecciones SQL. El coste de filtrado se traslada íntegramente a PostgreSQL, sin materializar subconjuntos intermedios de resultados en memoria.

4.2.2. Filtros de metadatos

Los filtros de metadatos permiten restringir los resultados de una colección en función de los atributos clave-valor asociados a sus entidades. En el contexto de TMDb, por ejemplo, esto equivale a operaciones como seleccionar únicamente películas del género *Action*, o excluir aquellas con idioma original distinto del inglés.

Cada filtro de inclusión se traduce en una subconsulta **EXISTS** sobre la tabla **metadata**. Si el usuario ha seleccionado múltiples valores para el mismo atributo, se genera una cláusula **EXISTS** independiente por cada valor, lo que impone que la entidad deba satisfacer *todos* ellos simultáneamente. Los filtros de exclusión (**notMfilters**) siguen la misma estructura pero con **NOT EXISTS**:

Listing 4.1: Filtros de metadatos

```

1 for (var entry : state.getMfilters().entrySet()) {
2     String attr = entry.getKey();
3     for (String val : entry.getValue()) {
4         sql.append(" AND EXISTS (SELECT 1 FROM metadata " + m
5             + " WHERE " + m + ".entity_id = " + e + ".
6                 global_id"
7             + " AND " + m + ".key = ? AND " + m + ".value = ?)
8                 ");
9         params.add(attr);
10        params.add(val);
11    }
12 }

```

Esta elección implica una semántica de conjunción estricta entre valores del mismo atributo, lo cual puede parecer contraintuitivo cuando los valores son mutuamente excluyentes — como ocurre con **Genres** en TMDb, donde una película puede pertenecer a varios géneros a la vez. En ese caso, seleccionar *Action* y *Drama* devuelve las películas que tienen *ambos* géneros, no las que tienen alguno de los dos. Esta semántica se mantuvo de forma deliberada para garantizar consistencia con el comportamiento del resto de filtros del sistema, y la describiremos como parte de los obstáculos encontrados en la sección 4.5.

4.2.3. Filtros de referencia

Los filtros de referencia permiten restringir los resultados de una colección en función de sus vínculos con entidades concretas de otra colección. En TMDb, esto cubre casos como mostrar únicamente las películas en las que una persona determinada participó como *Director*, o excluir películas con un *Actor* específico.

Cada filtro de referencia está identificado por tres elementos: la colección referenciada, el tipo de relación (*reason*) y el identificador de la entidad concreta. Al igual que con los metadatos, se genera una subconsulta **EXISTS** por cada combinación activa, realizando un **JOIN** entre la tabla **reference**, la tabla **entities** y la tabla **collections** para verificar la existencia de dicho vínculo:

Listing 4.2: Filtros de referencia

```

1 sql.append(" AND EXISTS ("
2     + "SELECT 1 FROM reference " + r
3     + " JOIN entities " + re + " ON " + r + ".target_id = " +
4       re + ".global_id"
5     + " JOIN collections " + rc + " ON " + re + ".
6       collection_global_id = "
7     + rc + ".global_id"
8     + " WHERE " + r + ".source_id = " + e + ".global_id"
9     + " AND " + rc + ".original_id = ?"
10    + " AND " + rc + ".dataset_id = ?"
11    + " AND " + r + ".reason = ?"
12    + " AND " + re + ".original_id = ?) ");

```

Los filtros de exclusión (`notRfilters`) utilizan NOT EXISTS con la misma estructura. Todos los filtros activos — de metadatos y de referencia, en sus variantes positiva o negativa — se aplican conjuntamente sobre la misma consulta base, de forma que el resultado final es la intersección de todas las condiciones establecidas por el usuario en ese momento.

4.2.4. Facetas: cómputo dinámico de valores disponibles

Las facetas son el mecanismo por el que el sistema informa al usuario de qué valores siguen siendo relevantes dado el contexto de filtrado actual: qué géneros tienen al menos una película entre los resultados visibles, qué directores aparecen en esas películas, o a qué cadenas están asociadas las series que quedan tras aplicar los filtros. Este cómputo es siempre dinámico: se recalcula en cada petición a partir del conjunto de entidades que satisfacen el estado actual.

El sistema distingue dos tipos de facetas, ambas servidas por `FacetsServlet` a través del endpoint `GET /api/facets`:

Las **facetas de metadatos** se obtienen mediante `getFacets()`, que construye una subconsulta con los identificadores de las entidades actualmente visibles — aplicando todos los filtros del estado — y sobre ella ejecuta un `GROUP BY` sobre la tabla `metadata`:

Listing 4.3: Cómputo dinámico

```

1 String sql =
2     "SELECT m.key, m.value, COUNT(*) as cnt "
3     + "FROM metadata m "
4     + "WHERE m.entity_id IN (" + subquery + ") "
5     + "GROUP BY m.key, m.value "
6     + "ORDER BY m.key, cnt DESC";

```

El resultado de la query se recorre fila a fila y se agrupa en un `LinkedHashMap` indexado por atributo (`key`). Por cada fila, se construye un mapa con tres campos: el valor del atributo (`value`), su frecuencia entre las entidades visibles (`count`), y un booleano `active` que indica si ese valor ya está seleccionado como filtro en el estado actual, que lo consulta mediante `isFilterActive()`. El uso de `LinkedHashMap` preserva el orden de inserción, que viene determinado por el `ORDER BY m.key, cnt`

DESC de la query: los atributos aparecen en orden alfabético y, dentro de cada atributo, los valores se listan de mayor a menor frecuencia. Este orden es el que el frontend recibe y renderiza directamente, sin necesidad de reordenar en el cliente.

Las **facetas de referencia** se obtienen mediante `getReferenceFacets()`, que sobre la misma subconsulta de entidades visibles realiza un `JOIN` con la tabla `reference` para descubrir qué entidades de otras colecciones están vinculadas a los resultados actuales, agrupadas por colección y tipo de relación. El resultado expone, por ejemplo, qué personas concretas aparecen como *Director* entre las películas visibles, junto con el número de películas en las que aparecen.

Junto a las facetas, el mismo endpoint devuelve los *links* disponibles — obtenidos mediante `getAvailableLinks()` — que indican a qué otras colecciones es posible navegar desde el conjunto actual, y los filtros activos en ese momento (`activeFilters`), permitiendo al frontend reflejar el estado de selección sin necesidad de mantener dicha información en el cliente.

4.3. Navegación transversal

Hasta ahora, hemos descrito los mecanismos de filtrado que operan dentro de una única colección: dadas las entidades de *Movies*, se restringe el conjunto según atributos propios o según vínculos con entidades concretas de otras colecciones. Entretanto, el sistema permite también *trasladarse* de una colección a otra siguiendo una relación, manteniendo y llevando el contexto de filtrado acumulado. Este mecanismo es lo que se denomina navegación transversal.

Damos un ejemplo concreto en TMDb: el usuario está explorando películas con el filtro de género *Action* activo. Desde ese contexto, decide navegar hacia la colección *People* siguiendo la relación *Actor*, para ver qué actores aparecen en esas películas de acción. Tras el salto, la colección activa pasa a ser *People*, pero el sistema recuerda que solo interesan las personas vinculadas al subconjunto de películas que cumplían el filtro establecido anteriormente. El historial de transiciones compone, por tanto, parte del contexto de exploración con responsabilidad expresa, no un simple registro de navegación.

4.3.1. El mecanismo de link y goback

La transición entre colecciones se realiza mediante la operación `link`, que recibe el identificador de la colección destino y el tipo de relación a seguir. Su implementación en `StateManager` tiene tres efectos:

Listing 4.4: Link

```
1 public void link(int env, String reason) {  
2     this.linkList.add(new Link(true, this.cur.  
3         getCurrentCollectionId(),  
4         reason, new State(this.cur)));  
5     this.cur.link(env, reason);  
6     this.cur.setLinks(new ArrayList<>(this.linkList));  
7 }
```

Primero, se añade un nuevo objeto `Link` al `linkList`, marcado como transición hacia delante (`forward = true`). Este objeto almacena la colección de origen, el tipo de relación seguida y una copia profunda del estado activo en el momento de la transición, preservando así los filtros que estaban activos antes del salto. Segundo, se actualiza la colección activa en `cur` a la colección destino. Tercero, se sincroniza la lista de links dentro del propio `State`, de forma que el `NavigationDAO` pueda accederla directamente al construir las queries.

La operación `goback` invierte la última transición registrada en el `linkList`. En lugar de eliminar el último `Link`, añade uno nuevo con la dirección invertida respecto al anterior —si el último era hacia delante, el nuevo es hacia atrás, y viceversa— y restaura como colección activa la colección de origen almacenada en ese `Link`:

Listing 4.5: Goback

```

1 public void goback() {
2     Link last = this.linkList.get(this.linkList.size() - 1);
3     if (last.isForward()) {
4         this.linkList.add(new Link(false, this.cur.
5                                 getCurrentCollectionId(),
6                                 last.getReason(), new State
7                                     (this.cur)));
8     } else {
9         this.linkList.add(new Link(true, this.cur.
10                                getCurrentCollectionId(),
11                                last.getReason(), new State
12                                    (this.cur)));
13     }
14     this.cur.goback(last.getEnv());
15     this.cur.setLinks(new ArrayList<>(this.linkList));
16 }

```

El hecho de que `goback` añada un nuevo `Link` en lugar de eliminar el último tiene una consecuencia relevante: el historial es siempre creciente y nunca se borra. Esto significa que la query SQL resultante debe recorrer toda la cadena de transiciones acumuladas, incluyendo los retrocesos, para reconstruir fielmente el contexto de exploración.

4.3.2. Subconsultas anidadas: traducción del historial de *links* a SQL

Cuando el estado contiene un historial de transiciones, `appendFilterClauses` no puede limitarse a aplicar los filtros del estado activo: debe también expresar en SQL la restricción que impone cada transición puesta anteriormente. Para ello utiliza una sobrecarga recursiva que recorre el `linkList` desde el último `Link` hacia el primero, generando en cada nivel una subconsulta anidada.

La estructura de la recursión tiene la siguiente definición: para cada nivel de profundidad, se aplican primero los filtros del estado en ese nivel mediante `appendStateFilters`, y a continuación se genera una cláusula `AND ... IN (SELECT ...)` que restringe las

entidades del nivel actual a aquellas que, a través de la relación registrada en el `Link`, están vinculadas a entidades del nivel siguiente. La dirección del `Link` determina qué columna de la tabla `reference` actúa como origen y cuál como destino:

Listing 4.6: Subconsultas anidadas

```

1  if (link.isForward()) {
2      sql.append(" AND " + e + ".global_id IN ("
3          + "SELECT " + r_link + ".target_id FROM reference " +
4              r_link
5          + " JOIN entities " + next_e + " ON " + r_link + ".
6              source_id = "
7          + next_e + ".global_id"
8          + " JOIN collections " + next_c + " ON " + next_e
9          + ".collection_global_id = " + next_c + ".global_id"
10         + " WHERE " + next_c + ".dataset_id = ? AND " + next_c
11         + ".original_id = ? AND " + r_link + ".reason = ? ");
12 } else {
13     sql.append(" AND " + e + ".global_id IN ("
14         + "SELECT " + r_link + ".source_id FROM reference " +
15             r_link
16         + " JOIN entities " + next_e + " ON " + r_link + ".
17             target_id = "
18         + next_e + ".global_id"
19         + " JOIN collections " + next_c + " ON " + next_e
20         + ".collection_global_id = " + next_c + ".global_id"
21         + " WHERE " + next_c + ".dataset_id = ? AND " + next_c
22         + ".original_id = ? AND " + r_link + ".reason = ? ");
23 }
24 appendFilterClauses(sql, params, link.getState(), linkList,
25     index - 1, depth + 1);
26 sql.append(") ");

```

Cada llamada recursiva abre una subconsulta que se cierra después de que la llamada interna haya completado su propio nivel, resultando en una estructura de paréntesis anidados que refleja correctamente la cadena de transiciones del usuario. Los identificadores de tabla (`e`, `e1`, `e2`, `r10`, `r11`...) se generan dinámicamente a partir del parámetro `depth` para evitar colisiones entre niveles.

Retomando el ejemplo anterior — películas de acción, navegación hacia *People* por la relación *Actor* — la query resultante para obtener los actores visibles tendría la siguiente forma:

Listing 4.7: Query resultante

```

1  SELECT DISTINCT e.original_id, e.name
2  FROM entities e
3  JOIN collections c ON e.collection_global_id = c.global_id
4  WHERE c.dataset_id = ? AND c.original_id = ?    -- People
5  AND e.global_id IN (
6      SELECT r10.source_id FROM reference r10
7      JOIN entities e1 ON r10.target_id = e1.global_id

```



```
8 JOIN collections c1 ON e1.collection_global_id = c1.  
   global_id  
9 WHERE c1.dataset_id = ? AND c1.original_id = ? -- Movies  
10 AND r10.reason = 'Actor'  
11 AND EXISTS (SELECT 1 FROM metadata m0 -- Genres = Action  
12              WHERE m0.entity_id = e1.global_id  
13              AND m0.key = ? AND m0.value = ?)  
14 )
```

La subconsulta interior opera sobre la colección *Movies* con sus filtros activos, y la consulta exterior recupera únicamente las personas de *People* que aparecen como origen de una relación *Actor* hacia alguna de esas películas.

4.4. Operaciones de conjuntos

Los mecanismos de filtrado y navegación transversal descritos hasta ahora operan siempre sobre un único contexto de exploración activo. Sin embargo, hay casos de uso que requieren combinar los resultados de dos o más contextos contruidos de manera independiente. Un posible ejemplo concreto en TMDb: el usuario quiere obtener todas las personas que han trabajado como *Director* en películas de acción o como *Actor* en series de ciencia ficción. No sabemos si hay intersección entre ambos conjuntos, y construir ambas condiciones simultáneamente dentro de un único estado sería impracticable, y posiblemente imposible con la semántica del filtrado descrito en la sección 4.2.

Para cubrir este caso, el sistema incorpora una operación de unión, donde el usuario puede guardar el contexto actual, iniciar una nueva exploración desde cero y, al consultar los resultados, obtener las entidades que satisfacen cualquiera de los contextos acumulados hasta entonces.

4.4.1. Construcción del unionSet

La operación `union` en `StateManager` tiene cuatro efectos sobre el estado interno:

Listing 4.8: Union

```
1 public void union(int collectionId) {  
2     this.cur.setLinks(new ArrayList<>(this.linkList));  
3     this.unionSet.add(this.cur);  
4     this.historyStack.clear();  
5     this.linkList.clear();  
6     this.cur = new State(this.datasetId, collectionId);  
7 }
```

Primero, sincroniza el historial de *links* dentro del estado activo, de forma que el estado guardado que se va a preservar tenga acceso a la cadena de transiciones completa. Segundo, añade el estado activo actual al `unionSet` sin realizar ninguna copia, dado que `cur` va a ser reemplazado inmediatamente, no hay riesgo de que futuras modificaciones lo alteren. Tercero, vacía tanto la `historyStack` como el `linkList`,

lo que disvincula completamente el nuevo contexto del anterior. Finalmente, inicializa `cur` con un estado limpio sobre la colección indicada, desde el que el usuario puede comenzar una exploración independiente.

El `unionSet` puede acumular un número arbitrario de entradas repitiendo esta operación. Cada entrada es un estado completo e independiente, con su propia colección activa, sus propios filtros y su propio historial de transiciones.

4.4.2. Resolución de la unión en base de datos

La consulta del `unionSet` se realiza a través del endpoint `GET /api/union`, servido por `UnionServlet`, que delega en los métodos `getUnionEntities()` y `countUnionEntities()` del `NavigationDAO`.

Ambos métodos construyen una única query que itera sobre todas las entradas del `unionSet` y concatena un bloque de condiciones por entrada, separados por `OR`:

Listing 4.9: Query de unión

```
1 for (int i = 0; i < unionSet.size(); i++) {  
2     State state = unionSet.get(i);  
3     if (i > 0) sql.append(" OR ");  
4     sql.append(" (c.dataset_id = ? AND c.original_id = ? ");  
5     params.add(state.getDatasetId());  
6     params.add(state.getCurrentCollectionId());  
7     appendFilterClauses(sql, params, state);  
8     sql.append(" ) ");  
9 }
```

Cada bloque es estructuralmente idéntico a la query que se generaría para ese estado en una consulta normal, aplicando `appendFilterClauses` sobre el estado correspondiente, lo que incluye sus filtros de metadatos, sus filtros de referencia y, si procede, las subconsultas anidadas derivadas de su historial de transiciones. La unión entre bloques se expresa con `OR` a nivel de la cláusula `WHERE`, y el `DISTINCT` en el `SELECT` garantiza que las entidades presentes en más de un bloque no aparezcan duplicadas en los resultados.

Una consecuencia de este diseño es que la unión puede combinar entradas sobre colecciones distintas. En el ejemplo anterior, el primer bloque opera sobre *People* filtrado por películas de acción con relación *Director*, y el segundo también sobre *People* pero filtrado por series de ciencia ficción con relación *Actor*. Que ambas entradas compartan colección activa no es un requisito del sistema: el campo `collection_id` devuelto por `getUnionEntities()` indica a qué colección pertenece cada entidad del resultado, permitiendo al frontend distinguirlas cuando fuera necesario.

4.5. Obstáculos encontrados y soluciones implementadas

A lo largo del desarrollo surgieron obstáculos de naturaleza diversa: algunos conceptuales, relacionados con la definición del modelo de datos y la lógica de na-

vegación; otros técnicos, derivados de las particularidades del entorno de ejecución. Esta sección recoge los más relevantes, describiendo el problema encontrado y la solución adoptada en cada caso.

Confusión conceptual en el prototipo inicial

El primer obstáculo fue de carácter conceptual y afectó al punto de partida del desarrollo. El modelo navegacional que articula el sistema — colecciones, referencias, filtros acumulados, transiciones — no tiene un equivalente directo en los sistemas de exploración de datos más habituales, lo que dificultó establecer desde el principio una representación interna clara.

El prototipo inicial no distinguía con precisión entre el estado de navegación del usuario y los datos del dominio, mezclando responsabilidades que en el diseño final quedaron separadas en capas distintas: el modelo (**State**, **StateManager**), la capa de acceso a datos (**NavigationDAO**) y los controladores. Esta confusión generó deuda técnica temprana que fue necesario resolver antes de poder implementar mecanismos más complejos como la navegación transversal o las operaciones de unión. La separación de responsabilidades que se describe a lo largo de este capítulo es, en parte, el resultado directo de haber identificado y corregido esas imprecisiones del prototipo.

Tamaño de las colecciones y trabajo con subconjuntos

TMDb expone colecciones de tamaño considerable: solo en películas y series, el catálogo completo supera con holgura las decenas de miles de entidades. Trabajar con el dataset completo durante el desarrollo habría introducido tiempos de carga, poblado de base de datos y ejecución de queries que habrían dificultado la iteración rápida.

La solución adoptada fue incorporar un mecanismo de filtrado por popularidad directamente en el script de procesamiento, controlado por las variables `FILTER_TOP_K_POPULAR` y `TOP_K_COUNT`:

```
1 FILTER_TOP_K_POPULAR = True
2 TOP_K_COUNT = 5000
```

Con este mecanismo activo, el script ordena las películas y series por su campo `popularity` de la API de TMDb y retiene únicamente las `TOP_K_COUNT` más populares antes de generar el dataset de salida. Esto permitió trabajar durante el desarrollo con un subconjunto representativo y manejable, manteniendo la opción de escalar al dataset completo simplemente desactivando el filtro o aumentando el límite.

Valores numéricos y de fecha en los metadatos: la decisión de usar rangos

Un obstáculo temprano en el diseño del filtrado fue la naturaleza de ciertos campos de metadatos en TMDb: presupuesto, recaudación, duración, fecha de estreno.

Almacenarlos como valores exactos en la tabla `metadata` y filtrar por igualdad exacta habría resultado inútil desde el punto de vista exploratorio: es improbable que un usuario quiera películas con exactamente 157 minutos de duración o con un presupuesto de exactamente 80.000.000 de dólares.

La solución fue discretizar estos valores en rangos durante el procesamiento, antes de insertarlos en la base de datos. Para valores numéricos, la función `bucket_range` agrupa cada valor en un intervalo de tamaño configurable:

```
1 def bucket_range(value, bucket_size):
2     bucket_start = int(num // bucket_size) * bucket_size
3     return f"{format_number(bucket_start)}-{"format_number(
        bucket_start + bucket_size - 1)}
```

Para fechas, se aplica una lógica análoga con granularidad configurable por año, mes y día, definida en `format.json` por colección. De este modo, el valor almacenado en `metadata` no es 157 sino 150-179, lo que permite al sistema de filtrado operar con igualdad de cadenas sobre rangos semánticamente útiles, sin necesidad de soportar operadores de comparación numérica en las queries. La alternativa de soportar expresiones regulares en los filtros de metadatos fue considerada y descartada: la complejidad añadida en la construcción de queries y la dificultad de exponerlo de forma intuitiva al usuario no compensaban el beneficio, siendo los rangos una solución más simple y suficientemente expresiva para los casos de uso identificados. Para los casos en que un usuario necesita combinar varios rangos, la operación de unión cubre esa necesidad de forma más predecible.

CORS y la comunicación entre frontend y backend

El sistema está organizado como dos procesos independientes: el frontend se sirve estáticamente y realiza peticiones al backend desplegado en Tomcat. Esta separación implica que las peticiones HTTP se originan desde un origen distinto al del servidor de la API, lo que activa la política de mismo origen (*same-origin policy*) del navegador y bloquea las respuestas por defecto.

El obstáculo no fue solo técnico sino también de comprensión: el mecanismo CORS (*Cross-Origin Resource Sharing*) no es transparente, y sus manifestaciones —peticiones bloqueadas sin mensaje de error claro, o peticiones `OPTIONS` inesperadas— no resultan inmediatamente evidentes si no se ha trabajado previamente con arquitecturas de frontend y backend separados. Fue necesario investigar el protocolo para entender que el navegador envía primero una petición de preflight con método `OPTIONS` antes de la petición real, y que el servidor debe responder con las cabeceras adecuadas para que el navegador autorice la comunicación.

La solución fue implementar un filtro de servlet, `CORSFilter`, que intercepta todas las peticiones mediante la anotación `@WebFilter(/*)` y añade las cabeceras necesarias en cada respuesta:

```
1 response.setHeader("Access-Control-Allow-Origin", origin);
2 response.setHeader("Access-Control-Allow-Credentials", "true")
3 ;
4 response.setHeader("Access-Control-Allow-Methods",
```

```
4     "GET, POST, PUT, DELETE, OPTIONS");
5
6     if ("OPTIONS".equalsIgnoreCase(request.getMethod())) {
7         response.setStatus(HttpServletResponse.SC_OK);
8         return;
9     }
10    chain.doFilter(req, res);
```

Un detalle relevante fue la combinación de `Access-Control-Allow-Credentials: true` con el reflejo dinámico del origen en lugar de un comodín *: los navegadores rechazan el uso de * cuando la petición incluye credenciales (en este caso, la cookie de sesión), por lo que el filtro lee el encabezado `Origin` de la petición y lo devuelve como valor explícito. Complementariamente, fue necesario configurar el `CookieProcessor` en `context.xml` con `sameSiteCookies="None"` para que la cookie de sesión se enviara correctamente en peticiones cross-origin.

El diseño de goback y sus implicaciones

La operación `goback` fue uno de los puntos del diseño que requirió más iteraciones. La dificultad radicó en que revertir una transición entre colecciones no es simplemente volver a la colección anterior: es volver al estado exacto que existía antes de la transición, incluyendo todos los filtros que estaban activos en ese momento.

Un primer enfoque consistió en eliminar el último `Link` del `linkList` al ejecutar `goback`, restaurando la colección activa a la del enlace eliminado. Este enfoque resultó problemático porque el historial perdía información: si el usuario navegaba hacia adelante, retrocedía y volvía a navegar, la cadena de transiciones no reflejaba fielmente el recorrido real, y las subconsultas generadas por `appendFilterClauses` producían resultados incorrectos.

La solución adoptada fue tratar `goback` de forma simétrica a `link`: en lugar de eliminar el último `Link`, se añade uno nuevo con la dirección invertida, preservando el historial completo de forma creciente:

```
1 public void goback() {
2     Link last = this.linkList.get(this.linkList.size() - 1);
3     if (last.isForward()) {
4         this.linkList.add(new Link(false,
5             this.cur.getCurrentCollectionId(),
6             last.getReason(), new State(this.cur)));
7     } else {
8         this.linkList.add(new Link(true,
9             this.cur.getCurrentCollectionId(),
10            last.getReason(), new State(this.cur)));
11    }
12    this.cur.goback(last.getEnv());
13    this.cur.setLinks(new ArrayList<>(this.linkList));
14 }
```

La consecuencia de este diseño, descrita en la sección 4.3, es que `appendFilterClauses` debe recorrer toda la cadena acumulada, incluidos los retrocesos, para construir co-

rectamente las subconsultas anidadas. El coste es una query potencialmente más larga conforme el usuario navega y retrocede, pero garantiza que el contexto de exploración se reconstruye de forma exacta en cada petición.

Capítulo 5

Pruebas y Validación

5.1. Casos de uso validados

Capítulo 6

Conclusiones y Trabajo Futuro

6.1. LLM como método de navegación

6.2. Distribuir el motor en varias máquinas

6.3. Cosas genéricas (usuario, historial, etc.)

Antes de la entrega de actas de cada convocatoria, en el plazo que se indica en el calendario de los trabajos de fin de grado, el estudiante entregará en el Campus Virtual la versión final de la memoria en PDF.

Introduction

Introduction to the subject area. This chapter contains the translation of Chapter 1.

Conclusions and Future Work

Conclusions and future lines of work. This chapter contains the translation of Chapter 6.

Contribuciones Personales

En caso de trabajos no unipersonales, cada participante indicará en la memoria su contribución al proyecto con una extensión de al menos dos páginas por cada uno de los participantes.

En caso de trabajo unipersonal, elimina esta página en el fichero `TFGTeXiS.tex` (comenta o borra la línea `\include{Capitulos/ContribucionesPersonales}`).

Estudiante 1

Al menos dos páginas con las contribuciones del estudiante 1.

Estudiante 2

Al menos dos páginas con las contribuciones del estudiante 2. En caso de que haya más estudiantes, copia y pega una de estas secciones.

Bibliografía

*Y así, del mucho leer y del poco dormir, se
le secó el cerebro de manera que vino a
perder el juicio.*
(modificar en Cascaras\bibliografia.tex)

Miguel de Cervantes Saavedra

BAUTISTA, T., OETIKER, T., PARTL, H., HYNA, I. y SCHLEGL, E. *Una Descripción de $\text{\LaTeX}$ 2 ϵ* . Versión electrónica, 1998.

KNUTH, D. E. *The $\text{\TeX}$ book*. Addison-Wesley Professional., 1986.

KRISHNAN, E., editor. *$\text{\LaTeX}$ Tutorials. A primer*. Indian $\text{\TeX}$ Users Group, 2003.

LAMPORT, L. *$\text{\LaTeX}$: A Document Preparation System, 2nd Edition*. Addison-Wesley Professional, 1994.

MITTELBACH, F., GOOSSENS, M., BRAAMS, J., CARLISLE, D. y ROWLEY, C. *The $\text{\LaTeX}$ Companion*. Addison-Wesley Professional, segunda edición, 2004.

OETIKER, T., PARTL, H., HYNA, I. y SCHLEGL, E. *The Not So Short Introduction to $\text{\LaTeX}$ 2 ϵ* . Versión electrónica, 1996.

Apéndice **A**

Título del Apéndice A

Los apéndices son secciones al final del documento en las que se agrega texto con el objetivo de ampliar los contenidos del documento principal.

Apéndice	B
----------	----------

Título del Apéndice B

Se pueden añadir los apéndices que se consideren oportunos.

Este texto se puede encontrar en el fichero Cascaras/fin.tex. Si deseas eliminarlo, basta con comentar la línea correspondiente al final del fichero TFGTeXiS.tex.

*–¿Qué te parece desto, Sancho? – Dijo Don Quijote –
Bien podrán los encantadores quitarme la ventura,
pero el esfuerzo y el ánimo, será imposible.*

*Segunda parte del Ingenioso Caballero
Don Quijote de la Mancha
Miguel de Cervantes*

*–Buena está – dijo Sancho –; fírmela vuestra merced.
–No es menester firmarla – dijo Don Quijote–,
sino solamente poner mi rúbrica.*

*Primera parte del Ingenioso Caballero
Don Quijote de la Mancha
Miguel de Cervantes*

